Tech Science Press

http://doi.org/10.32604/cmc.2026.000000

ARTICLE

Enhancing Splicing in Grey-box Fuzzing with Coverage-guided Target Selection and Adaptive Energy Scheduling

YoungIn Ahn[1], Minseok Lee[1], Soomin Kim [2,*] and Jaeseung Choi[1,*]

[1]Department of Computer Science and Engineering, Sogang University, Seoul, Republic of Korea
[2]Department of Computer Science and Engineering, Ewha Womans University, Seoul, Republic of Korea
*Corresponding Authors: Soomin Kim. Email: soomink@ewha.ac.kr, Jaeseung Choi. Email: jschoi22@sogang.ac.kr

Version November 4, 2026 submitted to Comput Mater Contin

ABSTRACT: Recently, coverage-based grey-box fuzz testing (fuzzing) has drawn growing attention as an effective technique for software security testing. Among its various components, input splicing combines parts of existing test cases to generate new test inputs. Despite its wide use in grey-box fuzzers, there has been little attention paid to the design choices of the splicing algorithm, and the seed to splice with is still chosen at random. In this paper, we first perform a statistical analysis of the splicing stage and find that the code coverage of a seed is positively associated with the usefulness of the resulting spliced seed. Based on this observation, we propose a novel input splicing algorithm that iteratively generates spliced seeds and evaluates their potential effectiveness to identify the most promising candidate for subsequent fuzzing. Additionally, we present an adaptive energy scheduling mechanism for the selected spliced seed to allocate the fuzzing budget according to its runtime characteristics. We implement our technique in SPAFL++, built on top of American Fuzzy Lop Plus Plus (AFL++), and perform a comprehensive evaluation on 9 open-source programs. SPAFL++ covered 1.64% more branches than AFL++, with statistically significant gains on four programs. Moreover, applying our technique to HavocMAB also increased branch coverage by 1.20%. These results show that the long-overlooked splicing stage is a practical avenue for improving grey-box fuzzing by complementing existing techniques.

KEYWORDS: software testing, fuzz testing, input splicing

1 Introduction

In the past decade, fuzz testing [1,2] (or fuzzing) has been actively explored as an effective technique for testing software and detecting bugs. While the core idea of fuzzing—automatically generating test inputs through random mutations—is straightforward, various fuzzing tools have been developed and have demonstrated their practical effectiveness by uncovering numerous real-world bugs [3]. Nowadays, fuzzing is widely adopted by software vendors to test their products during development [4].

Specifically, academic research has extensively focused on coverage-based grey-box fuzzing [5]. In this line of work, lightweight program instrumentation is used to collect code coverage feedback, which guides the evaluation of the usefulness of generated test cases. Over the past decade, grey-box fuzzing has been advanced in different ways, including improvements to seed scheduling algorithms [6,7], mutation strategies [8,9], and program execution efficiency [10,11].

One important component of grey-box fuzzing is *input splicing*, which combines parts of existing test cases to generate a new test case that may explore diverse execution behaviors. In particular, the splicing

© 2026 by the Authors. Submitted to Comput Mater Contin for under a Creative Commons Attribution 4.0 International License, which permitted unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

algorithm of American Fuzzy Lop (AFL) [3] has been inherited by numerous AFL-based fuzzers developed in subsequent studies, making it one of the most widely used implementations of input splicing in grey-box fuzzing. By generating unique test cases that simple random mutations alone cannot produce, splicing serves as a crucial component in enhancing the overall effectiveness of grey-box fuzzing.

Despite the widespread adoption of input splicing in grey-box fuzzing, its algorithmic design has received relatively little attention. Previous work on input splicing in grey-box fuzzing has mainly focused on finding effective splice positions by considering the input format structure [12,13]. In another line of work, learning-based approaches for identifying effective splicing positions without prior knowledge of the input format have been explored [14,15]. However, the selection of the inputs to be combined during the splicing operation has not been systematically studied in previous work and instead relies on random selection.

In this work, we propose a novel input splicing algorithm motivated by a statistical analysis of the conventional splicing stage. First, we perform a statistical analysis to identify the seed characteristics that are associated with the effectiveness of the splicing stage. Based on these findings, we devise an iterative strategy that generates and evaluates candidate spliced seeds to select the most promising one for subsequent fuzzing. Furthermore, our algorithm adaptively determines the energy (i.e., fuzzing effort to be spent) assigned to the selected spliced seed according to the runtime characteristics observed during its evaluation. Note that this approach is orthogonal to and can complement existing techniques for selecting effective splicing positions or improving other components of fuzzing.

We implement our splicing strategy in a system named SPAFL++ and evaluate it on 9 open-source programs. SPAFL++ outperformed the baseline AFL++ on 6 of the benchmark programs and achieved an overall branch coverage gain of 1.64%. These results show that our splicing strategy effectively improves the overall performance of grey-box fuzzing. To support open science, we make our implementation and evaluation framework publicly available at https://github.com/infosec-sogang/SPAFLPP and https://github.com/infosec-sogang/Splicing-Artifact.

The contributions of this paper can be summarized as follows:

- We conduct a statistical analysis of the conventional splicing stage in grey-box fuzzers to identify correlations between seed characteristics and the effectiveness of the splicing stage.
- We propose a novel input splicing algorithm that incorporates (1) iterative evaluation of candidate spliced seeds using a carefully designed scoring scheme, and (2) adaptive energy assignment for the selected spliced seed based on its runtime properties.
- We implement our technique in SPAFL++ and conduct a comprehensive evaluation on 9 benchmark programs to demonstrate its effectiveness.

The remainder of this paper is organized as follows. Section 2 provides background and reviews previous work. Section 3 presents our empirical analysis of the conventional splicing stage in grey-box fuzzing. Section 4 describes the design and implementation of our novel splicing strategy. Section 5 evaluates the effectiveness of our system. Section 6 discusses threats to validity and limitations. Lastly, Section 7 concludes the paper.

2 Background and Related Work

In this section, we first sketch the general grey-box fuzzing algorithm and discuss the role of the splicing operation in this algorithm. We then review previous work on grey-box fuzzing and discuss its limitations with respect to improving the algorithmic design of the splicing operation.

2.1 Coverage-based Grey-box Fuzzing

Coverage-based grey-box fuzzing has become the predominant paradigm for fuzzing due to its scalability and effectiveness. Black-box fuzzing [1,16] relies on random input generation without any knowledge of the program under test (PUT), so it is often inefficient in exploring the input space of complex programs. On the other hand, white-box fuzzing [17] uses dynamic symbolic execution to systematically explore the input space of the PUT, but it is often too slow to scale to large programs. To overcome the limitations of black-box and white-box fuzzing, grey-box fuzzing has been proposed as a middle ground that balances scalability and effectiveness, by using lightweight instrumentation to collect program knowledge, often in the form of code coverage information, to guide the generation of new test inputs.

Since the pioneering work of Böhme *et al.* [5], numerous studies have been conducted to advance coverage-based grey-box fuzzing. In these studies, AFL and its successor, AFL++, have been widely used as the baseline for implementing and evaluating new fuzzing techniques. For instance, there has been research on improving the seed scheduling strategy to adaptively allocate *energy* to favor seeds that reach rarely exercised paths while avoiding waste on frequently visited ones [5,7]. Other studies have improved the mutation strategy by learning to select and apply mutation operators that can yield more coverage-increasing inputs [9,18,19].

Despite these advances, most AFL-based fuzzers still follow the same high-level fuzzing workflow. Algorithm 1 presents the high-level pseudocode of the common grey-box fuzzing workflow shared by AFL and its derivatives. Fuzzing starts with an initial seed corpus S_0 and a time limit t. Before the fuzzing loop begins, Line 2 initializes the seed queue Q with S_0. The fuzzing loop continues until the time limit t is reached. After fuzzing is completed, Q is returned, containing all the interesting test cases discovered during fuzzing.

For each iteration of the fuzzing loop, a seed input s_1, which is used as the basis for generating new test cases, is first selected from Q (Line 4). Line 5 determines the energy e, which denotes the number of test cases to generate from the selected seed input s_1, by considering various features of the seed, such as its execution time. Then the two stages of fuzzing, the havoc stage (Lines 7–9) and the splicing stage (Lines 11–15), are performed. In both stages, new test cases tcs are generated through mutation and executed on the PUT. If they are deemed interesting (e.g., they trigger new code coverage), they are added to Q for future fuzzing iterations. Note that additional stages, such as deterministic mutation and trimming, are also included in the fuzzing process, but they are omitted from Algorithm 1 for brevity.

The havoc stage generates new test cases by applying random mutations to the selected seed input s_1. AFL defines a set of mutation operators, such as bit and byte flipping, arithmetic operations (adding or subtracting random integer values), and overwriting bytes with predefined interesting values (e.g., boundary values for 32-bit integers). Within `havoc_mutate()`, a random number of mutation operators is selected and cumulatively applied to generate a new test case tc.

While the havoc stage generates new test cases by mutating a single seed input s_1, the splicing stage generates new test cases by combining two seed inputs (s_1 and s_2). For a given seed input s_1, a target seed input s_2 is randomly selected from the seed queue Q. Then, splice positions (offsets) are randomly selected from s_1 and s_2, and the first part of s_1 and the second part of s_2 are concatenated to generate a spliced seed s_{spl}. Lastly, s_{spl} is further mutated using the previously described havoc mutation operators to generate new test cases.

Algorithm 1: Pseudo-algorithm of Grey-box Fuzzing Shared by AFL and its Derivatives.

```
function fuzz(S_0, t)
    Q ← initialize_queue(S_0)
    while time limit t not reached do
        s_1 ← pop_next(Q)
        e ← decide_energy(s_1)
        // Havoc stage
        for e times do
            tc ← havoc_mutate(s_1)
            run_and_save_if_interesting(tc, Q)
        // Splicing stage
        s_2 ← select_random(Q)
        s_spl ← splice_input(s_1, s_2)
        for e times do
            tc ← havoc_mutate(s_spl)
            run_and_save_if_interesting(tc, Q)
    return Q
```

2.2 Advances in Grey-box Fuzzing

A large number of works contribute to improving the seed scheduling strategy in coverage-based grey-box fuzzing. AFLFast [5] models grey-box fuzzing as a Markov chain and introduces power schedules that steer energy toward seeds exercising low-frequency paths, while AFLGo [20] adapts the schedule with simulated annealing to favor seeds closer to target locations. Entropic [21] reframes energy allocation from an information-theoretic perspective, assigning more energy to seeds that maximize the information gained about program behaviors, and EcoFuzz [7] casts the power schedule as a variant of the adversarial multi-armed bandit problem to adaptively curb energy wasted on high-frequency paths. Beyond energy allocation alone, several works refine seed prioritization with richer signals. Cerebro [22] jointly optimizes seed selection and power scheduling over multiple objectives such as code complexity and execution time, AFL-HIER [23] organizes seeds into a hierarchy and allocates energy with a reinforcement-learning scheduler, and K-Scheduler [24] estimates potential coverage gain of a seed via graph-centrality analysis over a graph derived from the inter-procedural control-flow graph (CFG). More recently, Qian *et al.* [25] observe that coverage-based seed selection discards seeds whose payloads are masked by unsatisfied coverage guards, and recover such seeds by progressively disabling those guards. However, all of these techniques assign energy to the original seeds in the queue, and do *not* consider the energy allocation for spliced seeds.

A line of work improves the mutation strategy in grey-box fuzzing, in particular which mutation operators to apply and in what order, rather than selecting them randomly as AFL does. MOpt [18] learns a selection probability for each mutation operator via particle swarm optimization, and DARWIN [26] continuously optimizes this operator distribution online with an evolution strategy without introducing user-tunable parameters. Instead of a single global distribution, other works make context-dependent choices. HavocMAB [27] models the per-operator choice in the havoc stage as a multi-armed bandit, SeamFuzz [28] clusters seeds by similarity and learns a per-cluster operator strategy via Thompson sampling, and MuoFuzz [19] conditions the next operator on the previously applied one to exploit interaction effects between operators. At the level of whole strategies rather than individual operators, xFUZZ [29] treats input- and mutation-scheduling strategies as independently switchable plugins and selects their composition

online via sliding-window Thompson sampling, while TrioFuzz [30] makes such adaptation low-latency by decoupling a centralized strategy learner from parallel fuzzing threads. Other works reshape what mutation does, rather than which operator is scheduled: Bulbasaur [31] generates branch-specific mutators online, IDFuzz [32] learns a neural mutation model to steer directed fuzzing toward target locations, and ZTaint-Havoc [33] reuses havoc mutations to infer which input bytes influence program behavior without extra executions. From a complementary angle, MendelFuzz [9] revisits the deterministic mutation stage that most modern fuzzers disable, showing that it can be made efficient by pruning inputs derived from seeds and byte offsets that rarely contribute new coverage. However, our work is orthogonal to these mutation-level techniques, as we focus on the splicing stage and how to select promising splicing targets rather than how inputs are mutated.

Coverage feedback is the core signal that guides fuzzing, and thus several works refine how coverage is measured. AFL and AFL++ rely on hit-count edge coverage collected through lightweight instrumentation, but this metric collapses distinct program behaviors that map to the same edges. Angora [34] augments edge coverage with calling-context information so that the same edge exercised under different call stacks is counted separately, while CollAFL [35] eliminates the hash collisions that cause the AFL coverage bitmap to conflate distinct edges. Ankou [36] scores an execution by the combination of branches it exercises, distinguishing inputs that share the same union of covered edges, and TortoiseFuzz [6] weights coverage by security-relevance to prioritize the more valuable inputs rather than treating all coverage equally. Xiao *et al.* [37] even replace code coverage with system call pattern feedback, enabling grey-box fuzzing of COTS binaries for which collecting code coverage is fragile or costly.

2.3 Input Splicing in Grey-box Fuzzing

We next discuss prior work on improving the splicing operation in grey-box fuzzing.

First, several studies develop effective strategies for choosing *where to splice* since splicing at arbitrary byte offsets may corrupt the syntactic structure of the input. AFLSmart [12] derives a virtual structure, a parse tree of chunks and attributes, from a user-supplied input specification. Nautilus [13] assumes a grammar and splices subtrees over derivation trees, and WEIZZ [38] infers tags from the processing logic of the PUT to locate chunk boundaries in unknown binary formats. GRIMOIRE [14] instead requires no format specification or grammar, generalizing coverage-increasing inputs by replacing coverage-irrelevant portions with gap tokens that then serve as candidate splicing positions. GSA-Fuzz [15] takes a learning-based approach, treating offset selection as a learning problem and using a gravitational search algorithm to bias splicing toward segments that have historically yielded coverage gains.

Another line of work investigates the contribution of splicing operations in fuzzing campaigns. Kukucka *et al.* [39] isolate the effect of each havoc mutator lever in AFL++ and toggle splicing and RedQueen [40] independently to study the contribution of each operator to coverage achievement. In protocol fuzzing, Zhang *et al.* [41] examine how splicing interacts with byte-level and region-level mutators in AFLNet, and report that splicing does not lead to statistically significant branch coverage gains overall, yet is particularly effective at reaching parser and error-handling branches.

Meanwhile, Fioraldi *et al.* [42] study the effectiveness of implementing the splicing operation as a mutation operator rather than a separate stage. Their results show that using splicing as a mutation operator can be more effective than implementing it as a stage in certain benchmarks. Based on this observation, the latest AFL++ implements both splicing modes and offers them to users.

Notably, none of the previous work has focused on the algorithmic design of the conventional splicing stage itself. In particular, we identify two important limitations in existing implementations of the splicing

Table 1: Benchmark programs used in the statistical analysis.

Program	Package	Version	Input Format	Command-line
c++filt	binutils	2.37	Raw string	*None* (standard input)
idlc	cyclonedds	53cf7c	Interface Definition Language (IDL)	input.idl
infotocap	ncurses	6.4	Text	input.dat
libpng_read_fuzzer†	libpng	69ddff	Portable Network Graphics (PNG)	input.png
objdump	binutils	2.37	Executable and Linkable Format (ELF)	-D input
readelf	binutils	2.37	Executable and Linkable Format (ELF)	-a input
tic	ncurses	6.4	Text	input.src
tiff_read_rgba_fuzzer†	libtiff	4.2.0	Tagged Image File (TIF)	input.tif
xml2_read_fuzzer†	libxml2	2.9.12	Extensible Markup Language (XML)	input.xml

† Harness programs provided by the OSS-Fuzz project [4].

stage. First, the target seed for the splicing operation is randomly selected, as shown in Line 11 of Algorithm 1. In this work, we aim to devise a more effective strategy for selecting promising splicing targets to enhance fuzzing performance. Second, at Line 13, the spliced seed s_{spl} reuses the energy computed for the original seed at Line 5. Therefore, if the spliced seed exhibits significantly different characteristics from the original seed s_1, the current algorithm cannot reflect these differences in energy scheduling. To address this limitation, we propose an adaptive energy scheduling strategy for spliced seeds.

To sum up, our study is orthogonal to previous work on the splicing stage, as we focus on devising a strategy for selecting promising splicing targets. Also, our strategy reduces redundant exploration of the search space by prioritizing spliced candidates that are likely to exercise new program behaviors, instead of treating all candidates equally. In addition, our algorithm indirectly alleviates the impact of bloated inputs generated during splicing, as it assigns less energy to slower-executing spliced seeds.

3 Analysis of Splicing Stage

We first perform an empirical analysis of the splicing stage implemented in AFL++ to better understand its behavior. The purpose of this study is to investigate which seeds lead to more effective fuzzing results when selected as splicing targets. Based on the findings presented in this section, we devise our novel splicing strategy in Section 4.

3.1 Experimental Setup

Our experiments were conducted on a server equipped with two Intel Xeon Gold 6426Y (2.5 GHz) central processing units (CPUs) and 512 GB of main memory. We used the splicing stage implemented in AFL++ 4.40c as the baseline. We chose AFL++ over AFL because it incorporates various state-of-the-art techniques that have been developed on top of AFL. During the experiment, the deterministic stage of AFL++ was disabled, following the common practice in fuzzing experiments [12,27,43,44].

Table 1 summarizes the 9 benchmark programs and their settings used for our study. We first included three programs from the binutils package, which were widely used in early grey-box fuzzing papers [5,20, 45,46]. In addition, we selected libraries for popular file formats—libpng, libtiff, and libxml2—which have been adopted as standard targets by modern fuzzing benchmark suites [9,19,39]. To include programs that process non-binary, textual inputs, we also selected idlc, infotocap, and tic from recent fuzzing studies whose evaluations span a variety of input formats [28,47,48]. Overall, our benchmark suite consists of programs that have been widely used in prior work and covers diverse input formats, including both

Table 2: Pearson correlation coefficients between each evaluated feature and the number of useful test cases generated from the spliced seed, reported in the mean ± standard deviation format.

Program	Bitmap Size of s_2	Bitmap Size of s_{spl}	Bitmap Similarity	Input similarity (Hamming)	Input similarity (Jaccard)
c++filt	0.250 ± 0.087	0.328 ± 0.092	-0.085 ± 0.050	0.110 ± 0.040	0.034 ± 0.103
idlc	-0.021 ± 0.024	0.073 ± 0.049	0.010 ± 0.022	-0.066 ± 0.050	0.052 ± 0.056
infotocap	0.104 ± 0.016	0.131 ± 0.038	0.042 ± 0.029	0.130 ± 0.062	0.171 ± 0.066
libpng_read_fuzzer	0.000 ± 0.054	0.274 ± 0.102	0.117 ± 0.052	-0.178 ± 0.039	-0.144 ± 0.071
objdump	0.294 ± 0.035	0.384 ± 0.062	0.266 ± 0.042	-0.045 ± 0.114	0.118 ± 0.122
readelf	0.232 ± 0.035	0.293 ± 0.053	0.100 ± 0.006	-0.126 ± 0.038	-0.012 ± 0.062
tic	0.099 ± 0.078	0.298 ± 0.143	0.036 ± 0.040	0.105 ± 0.035	0.165 ± 0.112
tiff_read_rgba_fuzzer	0.215 ± 0.044	0.384 ± 0.062	-0.049 ± 0.048	-0.141 ± 0.032	-0.153 ± 0.019
xml2_read_fuzzer	0.248 ± 0.078	0.279 ± 0.078	-0.136 ± 0.083	0.425 ± 0.167	0.189 ± 0.033
Average	0.158	0.272	0.033	0.024	0.047

unstructured inputs (raw string and text) and structured inputs. The initial seed corpus used for each program is publicly available in our artifact repository.

3.2 Statistical Analysis

To identify meaningful indicators of promising splicing targets, we conducted an exhaustive pairwise analysis of splicing. We first ran AFL++ on each program until either 1000 seeds had been added to the queue or three hours had elapsed. From the resulting seed queue, we randomly sampled 100 seed inputs. Note that we sampled 100 seeds from the queue rather than using the first 100 seeds added to the queue. The latter could contain many closely related inputs originating from the same parent seed, which would reduce the diversity of the samples. We limited the sample size to 100 because the number of possible pairs grows quadratically with the sample size, making exhaustive evaluation computationally expensive.

From the 100 sampled seed inputs, we evaluated the effectiveness of every possible splicing pair. For each ordered pair (s_1, s_2), we performed the splicing operation 10 times using randomly selected splice offsets. Then, each resulting spliced seed, which corresponds to s_{spl} in Algorithm 1, was fuzzed to generate 50 new test cases. We measured the effectiveness of a spliced seed by counting the number of generated test cases that were deemed interesting by AFL++ in terms of coverage gain. Due to the cumulative nature of coverage-based grey-box fuzzing, the order in which splicing pairs are evaluated can affect the measured effectiveness. Once a generated test case is added to the seed queue, subsequent test cases exercising the same coverage are no longer considered interesting. Therefore, we ensured that all ordered pairs were evaluated under the same coverage state.

We then computed the following features for each ordered pair (s_1, s_2): (1) the coverage bitmap size of s_2 (i.e., the number of edges covered by s_2), (2) the coverage bitmap size of the spliced seed, (3) the Jaccard similarity between the coverage bitmaps of s_1 and s_2, and (4) the similarity between the input bytes of s_1 and s_2, measured using either the Hamming distance or the n-gram-based Jaccard distance. We did not include the coverage bitmap size of s_1 because it is independent of the choice of s_2 and therefore cannot be used to select an effective splicing target.

To investigate which characteristics are associated with effective splicing pairs, we computed correlation coefficients between each evaluated feature and the number of interesting test cases generated from the spliced seed. Table 2 presents the Pearson correlation coefficients averaged over 10 repeated

Table 3: Spearman correlation coefficients between each evaluated feature and the number of useful test cases generated from the spliced seed, reported in the mean ± standard deviation format.

Program	Bitmap Size of s_2	Bitmap Size of s_{spl}	Bitmap Similarity	Input similarity (Hamming)	Input similarity (Jaccard)
c++filt	0.308 ± 0.114	0.394 ± 0.125	-0.079 ± 0.055	0.125 ± 0.061	0.005 ± 0.169
idlc	-0.033 ± 0.027	0.086 ± 0.052	0.013 ± 0.038	-0.044 ± 0.063	0.044 ± 0.053
infotocap	0.122 ± 0.025	0.138 ± 0.041	0.026 ± 0.026	0.148 ± 0.075	0.167 ± 0.066
libpng_read_fuzzer	0.004 ± 0.061	0.279 ± 0.124	0.132 ± 0.052	-0.215 ± 0.093	-0.181 ± 0.078
objdump	0.366 ± 0.033	0.477 ± 0.047	0.305 ± 0.060	-0.054 ± 0.107	0.100 ± 0.130
readelf	0.255 ± 0.038	0.305 ± 0.037	0.074 ± 0.027	-0.087 ± 0.038	0.029 ± 0.060
tic	0.091 ± 0.086	0.265 ± 0.109	0.039 ± 0.039	0.128 ± 0.041	0.179 ± 0.133
tiff_read_rgba_fuzzer	0.201 ± 0.043	0.444 ± 0.109	-0.091 ± 0.049	-0.253 ± 0.025	-0.161 ± 0.024
xml2_read_fuzzer	0.177 ± 0.075	0.437 ± 0.074	-0.020 ± 0.100	0.214 ± 0.051	0.117 ± 0.044
Average	0.166	0.314	0.044	-0.004	0.033

experiments, whereas Table 3 presents the Spearman correlation coefficients averaged in the same manner. For both correlation measures, the coverage bitmap sizes of s_2 and s_{spl} exhibited relatively stronger correlations with splicing effectiveness than the other features in most programs. In particular, the coverage bitmap size of the spliced seed (s_{spl}) showed the highest coefficients on average among the examined features. These results suggest that a wider range of execution behaviors exercised by a seed is often inherited during splicing and subsequent mutations, leading to more effective operation of the splicing stage.

The input byte similarity between the two seeds used for splicing also showed meaningful correlations for certain benchmark programs. Specifically, the Hamming distance between the two seeds exhibited the strongest correlation in xml2_read_fuzzer. However, both the magnitude and direction of these correlations varied considerably across the benchmark programs. Therefore, input similarity does not provide a stable basis for selecting effective splicing targets in general.

4 Methodology

In this section, we present our novel design of splicing algorithm motivated by the findings from Section 3.

4.1 Overview

Fig. 1 provides an overview of our proposed splicing strategy. Our approach repeatedly selects a target seed for splicing and evaluates the spliced seed to identify the most promising candidate. It then adaptively determines the energy assigned to the selected spliced seed before applying havoc mutations to generate test cases. Algorithm 2 presents the overall grey-box fuzzing workflow extended with our novel splicing strategy in more detail. Specifically, Lines 11–19 introduce two key modifications to the conventional splicing stage. First, in Lines 12–18, our splicing algorithm iteratively generates and evaluates multiple candidate spliced seeds to identify the most promising one for subsequent test case generation with havoc mutation at Line 20. The details of our iterative splicing process are provided in Section 4.2. Second, we compute an appropriate energy for the spliced seed by considering its characteristics at Line 19, unlike Algorithm 1 that reuses the energy computed for the original seed s_1. Section 4.3 further describes the design of our adaptive scheduling strategy.

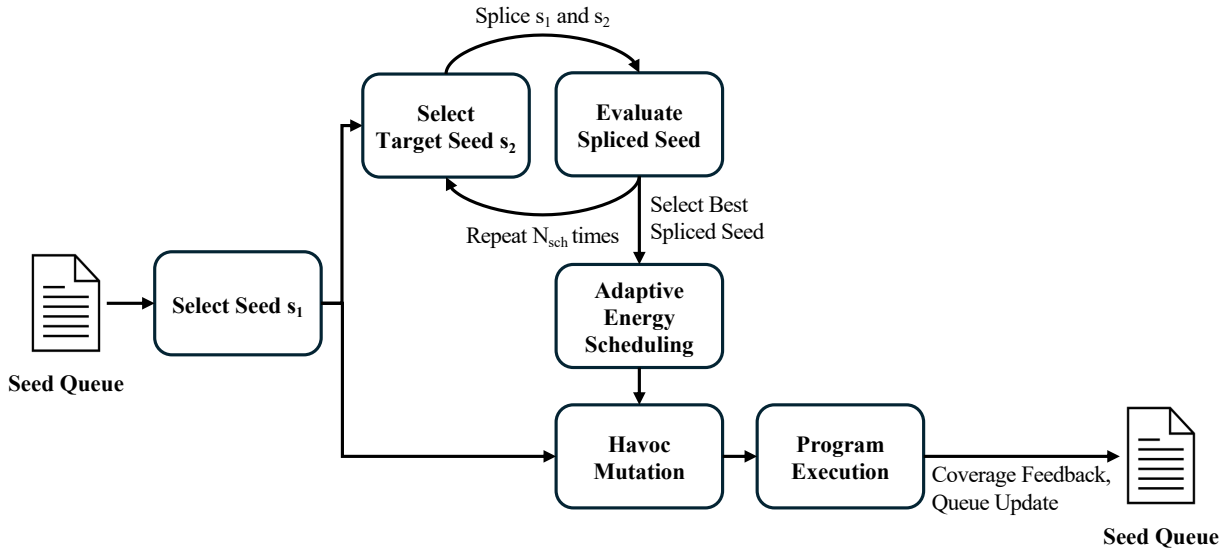


Figure 1: Overall workflow of SPAFL++.

4.2 Selection and Evaluation of Splicing

In our algorithm, the loop at Line 12 is iterated N_{sch} times to splice a seed, evaluate its usefulness, and choose the best spliced seed among the candidates. N_{sch} is the configuration parameter of our methodology, and the additional computational cost incurred by our splicing strategy grows linearly with N_{sch}, resulting in $O(N_{sch})$ time complexity. Our implementation currently uses 8 as the default value. In Section 5.4, we study the impact of this parameter on the overall effectiveness of fuzzing and justify our parameter choice.

For each trial, the target seed for splicing, s_2 is selected from the queue with a probability proportional to its coverage bitmap size (Line 13). This design is based on the relatively strong correlation between the coverage bitmap size of a seed and the effectiveness of the resulting spliced seed observed in Section 3.2. By assigning higher selection probabilities to seeds with larger coverage bitmaps, we increase the likelihood of choosing more promising seeds for splicing.

Next, the chosen s_2 is spliced with s_1, and the resulting seed is executed on the target program at Line 15 to obtain its coverage bitmap and execution time. During this execution, we also check whether the spliced seed itself is an interesting test case that increases code coverage, and add it to the seed queue if needed. Then, the obtained information is used to compute a score for the spliced seed. This score is used to select the most promising spliced seed to be passed to the subsequent loop at Line 20. The score is computed with the following formula:

$$\text{score}(s) = \frac{\text{cov}(s)/\text{cov}_{\text{avg}}}{\max(1, \lfloor \text{time}(s)/\text{time}_{\text{avg}} \rfloor)} \quad (1)$$

where cov_{avg} denotes the average coverage bitmap size of the seeds stored in the queue at the time of evaluation. Similarly, time_{avg} denotes the average execution time of the seeds currently stored in the queue.

In Eq. 1, we use the coverage bitmap size of the spliced seed as the primary factor for selecting the most promising candidate. Although the coverage bitmap size of s_2 is a useful indicator that can be obtained before executing the spliced seed, the coverage bitmap size of the spliced seed has a stronger correlation with its effectiveness, as discussed in Section 3.2. Therefore, after executing the spliced seed, we use its

Algorithm 2: Grey-box fuzzing with our novel splicing algorithm.

```
1  function fuzz(S_0, t, N_sch)
2      Q ← initialize_queue(S_0)
3      while time limit t not reached do
4          s_1 ← pop_next(Q)
5          e ← decide_energy(s_1)
6          // Havoc stage
7          for e times do
8              tc ← havoc_mutate(s_1)
9              run_and_save_if_interesting(tc, Q)
10         // Splicing stage with our new design
11         best_score ← −1
12         for N_sch times do
13             s_2 ← weighted_select(Q)
14             s' ← splice_input(s_1, s_2)
15             score ← run_and_evaluate(s', Q)
16             if score > best_score then
17                 best_score ← score
18                 s_spl ← s'
19         e_spl ← decide_energy_for_spliced(s_spl)
20         for e_spl times do
21             tc ← havoc_mutate(s_spl)
22             run_and_save_if_interesting(tc, Q)
23     return Q
```

actual coverage bitmap size to evaluate its quality more directly. The numerator is computed by normalizing this value with the average coverage bitmap size of the seeds currently stored in the queue.

At the same time, we use the normalized execution time of the spliced seed in the denominator of Eq. 1. During our fuzzing experiments, we found that spliced seeds with large coverage often exhibit significantly longer execution times on certain benchmarks. As a result, simply selecting seeds with large coverage can decrease the execution throughput of fuzzing and eventually degrade its overall effectiveness. Therefore, we penalize slower spliced seeds by considering their execution time. We use the floor operator to quantize this penalty, so that a seed is penalized only when its execution time exceeds an integer multiple of the average execution time. This prevents small differences in execution time from affecting the ranking of spliced seeds. In addition, the max operator prevents favoring seeds that execute faster than the average seed. Our goal is to prioritize seeds with large coverage while avoiding excessively slow seeds, rather than favoring fast-executing seeds.

4.3 Energy Scheduling for Spliced Seed

Line 19 in Algorithm 2 determines the energy (i.e., fuzzing effort to be spent) for the selected spliced seed. During this computation, we reuse the execution time and coverage bitmap size collected during the candidate evaluation described in Section 4.2. Specifically, the energy is computed with the following formula:

$$\text{energy}(s) = \frac{\text{cov}(s)}{\text{cov}_{\text{avg}}} \times \frac{\text{time}_{\text{avg}}}{\text{time}(s)} \times k \tag{2}$$

where k is a scaling coefficient that controls the overall execution budget. In our system, we use the same coefficient value with the baseline fuzzer, to ensure a fair comparison. Section 4.4 provides more details at the implementation level.

The purpose of Eq. 2 is to assign more computational resources to the seeds that execute faster and have higher coverage, thereby maximizing the expected benefit of fuzzing them. Unlike Eq. 1, the execution-time factor in Eq. 2 is not quantized with the floor operator, to account for subtle differences in execution time of the spliced seeds and enable finer-grained scheduling.

Although using coverage bitmap size and execution time in energy scheduling is a common strategy [3, 49], our novelty lies in using the characteristics of the spliced seed, not the original seed, for energy assignment in the splicing stage. In existing fuzzers based on AFL and AFL++, the spliced seed is not executed on the target program, which makes this design not applicable. Meanwhile, our iterative evaluation of spliced seeds naturally provides the runtime characteristics of the selected spliced seed before energy scheduling, making the proposed idea practical without introducing additional program executions.

4.4 Implementation

We implemented our technique in a system called SPAFL++, which is developed on top of AFL++ 4.40c [50]. We replaced AFL++'s original splicing procedure with our score-based spliced seed selection in Section 4.2 and adaptive energy scheduling for the selected seed in Section 4.3. We left all other components of AFL++ unchanged, including the coverage measurement, seed queue management, and energy scheduling in the havoc stage. For energy scheduling in the splicing stage, we used the base energy computed by AFL++ as the scaling factor k used in Eq. 2. In other words, we replaced only the factor computed from the original seed (s_1 in Algorithm 1) with a factor computed from the spliced seed, as described in Eq. 2. Then, the resulting energy value is rounded to an integer and clipped to the same minimum and maximum bounds used by AFL++.

In Section 5.2, we also implement our splicing technique on top of another baseline fuzzer, HavocMAB [27], and evaluate it in terms of code coverage and the number of bugs found. In this implementation, we similarly reuse all components of the baseline fuzzer except its splicing stage.

5 Evaluation

In this section, we perform experiments to answer the following research questions (RQs).

RQ1. Can our splicing strategy improve code coverage and bug-finding effectiveness of baseline fuzzers? (Section 5.2)

RQ2. How does our technique affect the effectiveness and efficiency of the splicing stage? (Section 5.3)

RQ3. How does the configuration parameter (N_{sch}) affect the performance of our technique? (Section 5.4)

5.1 Experimental Setup

All experiments were conducted on three servers with identical hardware configurations, each equipped with two Intel Xeon Gold 6526Y (2.8 GHz) CPUs and 256 GB of main memory, running Ubuntu 22.04. We ran 60 Docker containers concurrently on each server, each responsible for a single fuzzing campaign. We assigned a dedicated CPU core to each container and limited its memory usage to 4 GB. We ran each fuzzing campaign for 24 hours and repeated each experiment 20 times to account for the randomness inherent in fuzzing.

Table 4: Comparison between AFL++, SPAFL++, HavocMAB, and HavocMAB combined with our technique in terms of branch coverage (Cov.) and the number of unique bugs found (Bug #). Numbers are reported in mean ± standard deviation format, and the highest coverage value in each row is highlighted in bold.

Program	AFL++		SPAFL++		HavocMAB		HavocMAB + Ours	
	Cov.	Bug #	Cov.	Bug #	Cov.	Bug #	Cov.	Bug #
c++filt	**2550** ± 48	2.2 ± 0.5	2539 ± 37	2.4 ± 0.5	2504 ± 33	1.8 ± 0.8	2492 ± 52	1.7 ± 0.8
idlc	6076 ± 320	18.5 ± 0.6	**6244** ± 352	19.0 ± 0.6	6144 ± 344	17.9 ± 0.9	6213 ± 341	17.5 ± 1.9
infotocap	1931 ± 220	0.2 ± 0.4	2040 ± 240	0.4 ± 0.6	2303 ± 236	1.1 ± 0.7	**2447** ± 115	1.2 ± 0.4
libpng_read_fuzzer	**1602** ± 4	6.0 ± 0.0	**1602** ± 3	6.0 ± 0.0	1552 ± 39	6.0 ± 0.0	1565 ± 41	6.0 ± 0.0
objdump	4872 ± 427	0.2 ± 0.4	4901 ± 319	0.4 ± 0.5	5481 ± 147	0.1 ± 0.3	**5484** ± 121	0.1 ± 0.3
readelf	12373 ± 132	0.0 ± 0.0	**12758** ± 414	0.0 ± 0.0	12366 ± 302	0.0 ± 0.0	12616 ± 316	0.0 ± 0.0
tic	1733 ± 22	1.0 ± 0.0	1733 ± 18	1.0 ± 0.0	**1815** ± 29	1.0 ± 0.0	1794 ± 25	1.0 ± 0.0
tiff_read_rgba_fuzzer	5221 ± 128	0.0 ± 0.0	**5320** ± 127	0.0 ± 0.0	5077 ± 277	0.0 ± 0.0	5184 ± 166	0.0 ± 0.0
xml2_read_fuzzer	13190 ± 52	0.0 ± 0.0	**13224** ± 65	0.0 ± 0.0	13099 ± 62	0.0 ± 0.0	13148 ± 53	0.0 ± 0.0
Total	49548	28.1	50361	29.2	50341	27.9	50943	27.5

To evaluate the effectiveness of our technique, we used two baseline fuzzers. We used AFL++ 4.40c as our first baseline for comparison, since SPAFL++ is built on top of it. We note that AFL++ has been widely used as the primary baseline in recent fuzzing research to assess the isolated effect of novel fuzzing techniques [9,19,51]. Additionally, we used HavocMAB [27], which improves the havoc stage of grey-box fuzzing, as another baseline. This allows us to demonstrate that our technique is orthogonal to existing approaches that target other aspects of fuzzing. To this end, we implemented an enhanced version of HavocMAB that incorporates our splicing technique. For all fuzzers used in the experiments, we disabled the deterministic stage and enabled the splicing stage to ensure a fair comparison. In Sections 5.2 and 5.3, we ran our splicing algorithm with the default configuration parameter (N_{sch} = 8) introduced in Section 4.2.

We conducted the experiments on the same 9 benchmark programs used in our empirical study (Table 1), using the same initial seed corpora and command-line options. We measured the number of branches covered and the number of unique bugs found by each fuzzer, and reported the mean and standard deviation over the 20 trials. To classify crashes and count unique bugs, we adopted the patch-based triage method suggested by Klees *et al.* [46]. Specifically, we replayed every crashing input on the patched versions of each program, and grouped all crashes eliminated by the same patch as a single bug.

5.2 Comparison with Baseline Fuzzers

To evaluate the overall effectiveness of our technique, we compared AFL++, SPAFL++, HavocMAB, and HavocMAB integrated with our technique (HavocMAB + Ours) in terms of branch coverage and the number of unique bugs found. Table 4 reports the mean and standard deviation of both metrics computed over 20 repetitions of 24-hour fuzzing campaigns. Table 5 presents an in-depth coverage comparison between the two baselines and their counterparts enhanced with our splicing strategy: it reports the relative coverage improvement that our technique achieves over each base fuzzer, along with the p-value of a one-sided Mann–Whitney U test and the Vargha–Delaney $\hat{A}_{12}$ effect size. We did not perform statistical analysis on the number of bugs found, as the bug counts exhibited little or no variation across repeated trials and were largely similar across fuzzers for many benchmark programs.

In terms of branch coverage, SPAFL++ outperformed AFL++ on 6 of the 9 programs and increased the total coverage by 1.64%, from 49548 to 50361 branches. On idlc, infotocap, readelf, and xml2_read_fuzzer, the improvements were statistically significant, with p-values below 0.05 and $\hat{A}_{12}$ values ranging from 0.69

Table 5: Relative branch coverage improvement (+Cov) obtained by adding our technique to each baseline fuzzer (AFL++ and HavocMAB), along with the p-value of a one-sided Mann–Whitney U test and the Vargha–Delaney $\hat{A}_{12}$ effect size [52]. Comparison results that are statistically significant ($p \leq 0.05$) with a non-negligible effect size ($\hat{A}_{12} \geq 0.56$) are highlighted in bold.

Program	AFL++ vs. SPAFL++			HavocMAB vs. HavocMAB + Ours		
	+Cov (%)	p-value	$\hat{A}_{12}$	+Cov (%)	p-value	$\hat{A}_{12}$
c++filt	−0.43	0.9030	0.3812	−0.48	0.7796	0.4300
idlc	**+2.76**	**0.0212**	**0.6913**	+1.12	0.0599	0.6450
infotocap	**+5.64**	**0.0192**	**0.6925**	**+6.25**	**0.0036**	**0.7500**
libpng_read_fuzzer	0.00	0.6077	0.4763	**+0.84**	**0.0467**	**0.6562**
objdump	+0.60	0.2989	0.5500	+0.05	0.5054	0.5000
readelf	**+3.11**	**0.0012**	**0.7825**	**+2.02**	**0.0004**	**0.8100**
tic	0.00	0.5000	0.5012	−1.16	0.9848	0.3013
tiff_read_rgba_fuzzer	+1.90	0.0552	0.6488	+2.11	0.1552	0.5950
xml2_read_fuzzer	**+0.26**	**0.0199**	**0.6887**	**+0.37**	**0.0083**	**0.7225**

to 0.78. We observed a similar trend with HavocMAB, where HavocMAB + Ours achieved higher coverage than HavocMAB on 7 programs and raised the total number of covered branches from 50341 to 50943, an increase of 1.20%. As in the comparison with AFL++, the gains on four programs, namely infotocap, libpng_read_fuzzer, readelf, and xml2_read_fuzzer, were statistically significant with non-negligible effect sizes. These results suggest that our technique can improve the code coverage of grey-box fuzzing without being tied to a specific baseline fuzzer. Compared with prior studies on input splicing [12,13,38], our work demonstrates that improving the selection of splicing targets and energy scheduling of resulting spliced seeds can also yield a meaningful coverage gain. Moreover, the improvement observed in HavocMAB + Ours suggests that improving the splicing stage can provide complementary benefits to the techniques that optimize the havoc stage.

To further examine how branch coverage evolves over time, Fig. 2 plots the average branch coverage of the four fuzzers throughout the 24-hour fuzzing campaign. SPAFL++ stayed ahead of AFL++ throughout the campaign on idlc, tiff_read_rgba_fuzzer, and xml2_read_fuzzer, and on readelf it surpassed all the other fuzzers after 12 hours and finished with the highest coverage. Likewise, HavocMAB + Ours maintained its lead over HavocMAB for the entire 24 hours on every program except c++filt, where it fell slightly behind, and objdump and tic, where the two curves largely overlapped. We also found a notable case on infotocap, where SPAFL++ and HavocMAB + Ours each made a distinct jump in coverage while their base fuzzers had already plateaued. This jump suggests that our splicing strategy keeps uncovering new branches even after the base fuzzers stagnate.

Regarding bug-finding capability, SPAFL++ detected more unique bugs than AFL++ on c++filt, idlc, infotocap, and objdump and the same number on the remaining 5 programs, raising the total from 28.1 to 29.2. Note that SPAFL++ found more bugs on c++filt even though it covered slightly fewer branches than AFL++. HavocMAB + Ours found more bugs than HavocMAB on infotocap and the same number on 6 programs, while it found marginally fewer on c++filt and idlc. Overall, the total mean bug count remained comparable, decreasing slightly from 27.9 to 27.5. On idlc, standard deviations of bug counts were relatively higher than those of other programs for both fuzzers, indicating greater variability in bug discovery across individual trials.

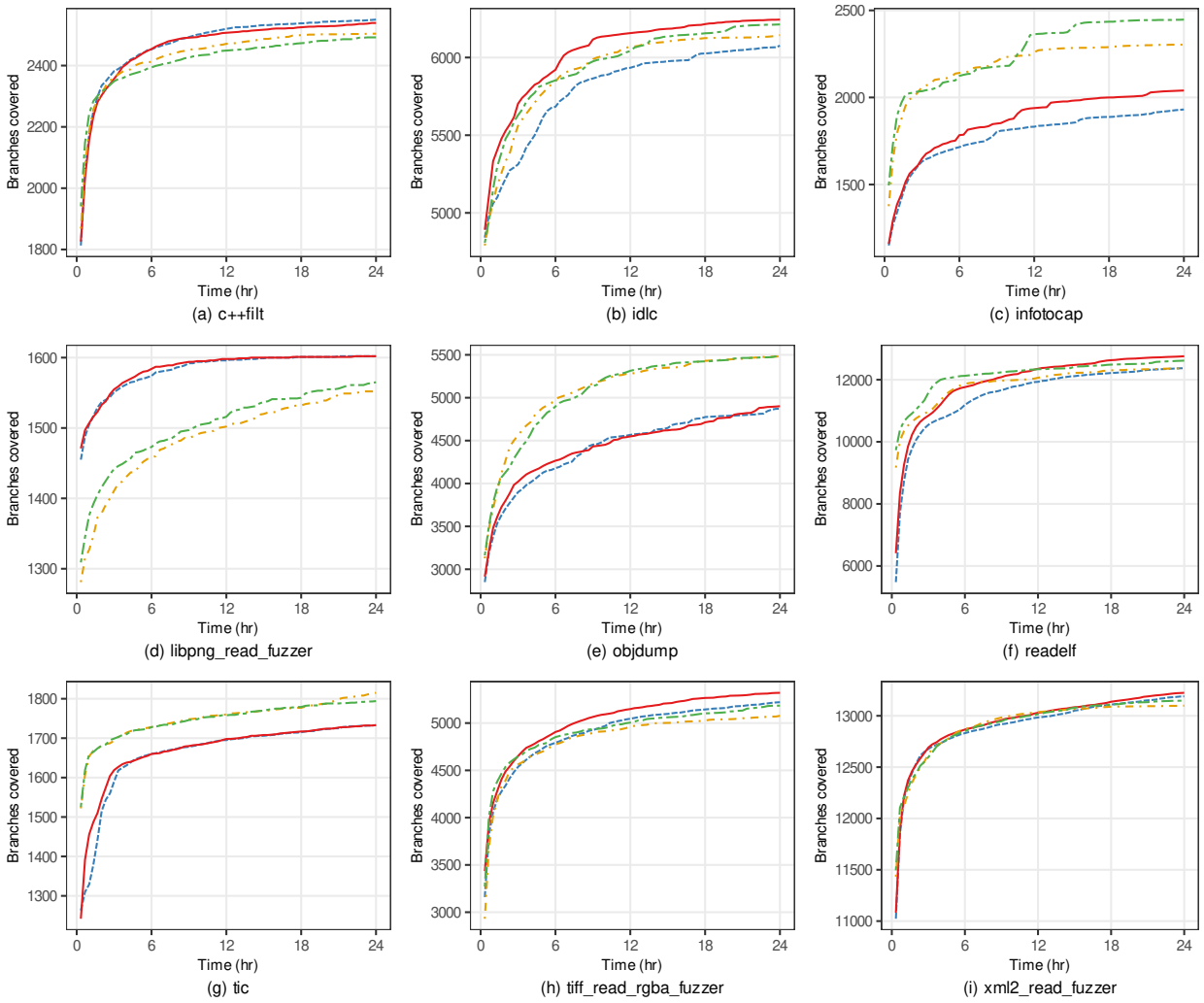


Figure 2: Average branch coverage of AFL++, SPAFL++, HavocMAB, and HavocMAB + Ours over time for each of the 9 benchmark programs: (a) c++filt, (b) idlc, (c) infotocap, (d) libpng_read_fuzzer, (e) objdump, (f) readelf, (g) tic, (h) tiff_read_rgba_fuzzer, and (i) xml2_read_fuzzer. (AFL++, SPAFL++, HavocMAB, HavocMAB+Ours)

5.3 Effectiveness and Efficiency of the Splicing Stage

In this section, we further investigate how each component of our novel splicing strategy affects the fuzzing effectiveness and execution efficiency of the splicing stage. To measure the effectiveness of the splicing stage, we divided the number of interesting test cases found during this stage by the total number of executions performed in it. To measure the efficiency of the splicing stage, we used the execution throughput of this stage, which is computed as the number of executions per second.

Table 6 presents the effectiveness and efficiency of the splicing stage for AFL++, SPAFL++, and two intermediate variants of SPAFL++ introduced for an ablation study. The first variant only performs weighted splicing target selection based on coverage bitmap size and does not incorporate the remaining components of our splicing strategy. The second variant implements iterative evaluation of splicing candidates on top of the first variant, but leaves the energy scheduling scheme unchanged. As summarized in the last row of the table, the overall splicing effectiveness and branch coverage progressively increase as the components of our splicing algorithm are gradually introduced.

Table 6: Comparison of AFL++, two intermediate variants of SPAFL++, and full SPAFL++ in terms of branch coverage (Cov.), splicing effectiveness (Eff.), and execution throughput (Exec.).

Program	AFL++			SPAFL++ (Weighted Sel.)			SPAFL++ (Eval. Spliced)			SPAFL++ (Full)		
	Cov.	Eff.	Exec.	Cov.	Eff.	Exec.	Cov.	Eff.	Exec.	Cov.	Eff.	Exec.
c++filt	2550	0.019	353.4	2525	0.025	282.9	2501	0.025	255.2	2539	0.025	268.0
idlc	6076	0.013	172.0	6281	0.016	151.3	6071	0.020	163.9	6244	0.021	163.9
infotocap	1931	0.017	129.8	1938	0.017	128.2	1979	0.024	112.7	2040	0.022	115.1
libpng_read_fuzzer	1602	0.002	243.9	1605	0.002	289.9	1604	0.002	275.5	1602	0.002	280.9
objdump	4872	0.015	237.9	4877	0.015	247.1	4759	0.020	240.3	4901	0.019	257.2
readelf	12373	0.085	280.6	12244	0.084	282.5	12566	0.110	272.8	12758	0.136	272.6
tic	1733	0.009	194.6	1718	0.009	201.3	1729	0.013	165.8	1733	0.013	167.6
tiff_read_rgba_fuzzer	5221	0.011	272.3	5288	0.011	276.7	5290	0.016	250.4	5320	0.017	255.4
xml2_read_fuzzer	13190	0.030	213.8	13150	0.033	189.9	13223	0.044	197.7	13224	0.047	198.3
Average	5505	0.022	233.1	5514	0.023	227.8	5525	0.031	214.9	5596	0.034	219.9

Meanwhile, we note that the execution throughput of the splicing stage substantially decreases in the second variant of SPAFL++ for certain benchmark programs. This slowdown suggests that prioritization of seeds with large coverage bitmaps can occasionally favor seeds with slow execution speeds. However, the full-fledged version of SPAFL++ partially mitigates this throughput degradation by assigning lower energy to slower-executing seeds. To sum up, our splicing strategy improves overall fuzzing performance over the baseline by striking a good balance between the effectiveness and efficiency of the splicing stage.

5.4 Impact of Configuration Parameter

The N_{sch} parameter from Section 4.2 controls how many splicing targets SPAFL++ examines in each splicing stage before selecting the best candidate. This parameter governs a trade-off between the cost of evaluating candidates and the quality of the selected spliced seed. With a small N_{sch}, SPAFL++ may not examine enough candidates to discover a high-quality splicing result. Conversely, with a large N_{sch}, SPAFL++ may find a better candidate but may consume too much execution budget that could otherwise be used for havoc mutation.

To evaluate this trade-off, we measured the total branch coverage of SPAFL++ and the proportion of fuzzing time spent on candidate evaluation on the 9 benchmark programs while varying N_{sch} over 2, 4, 8, and 16, keeping all other settings identical to those in Section 5.1. Fig. 3 shows that SPAFL++ covered only 49583 branches with $N_{sch} = 2$, marginally above the 49548 branches of AFL++. Increasing N_{sch} to 4 and 8 progressively improved the coverage to 50142 and 50361 branches, respectively. However, when N_{sch} was increased further, the cost of evaluating candidates outweighed its benefit. With $N_{sch} = 16$, SPAFL++ spent 32.2% of its fuzzing time on candidate evaluation on average, compared to 5.8%, 10.8%, and 19.4% for N_{sch} of 2, 4, and 8, respectively, and the coverage dropped to 49817 branches, losing most of the gain achieved at $N_{sch} = 8$. Overall, N_{sch} values of 4 and 8 offered the best balance, with 8 achieving the highest branch coverage among the tested values.

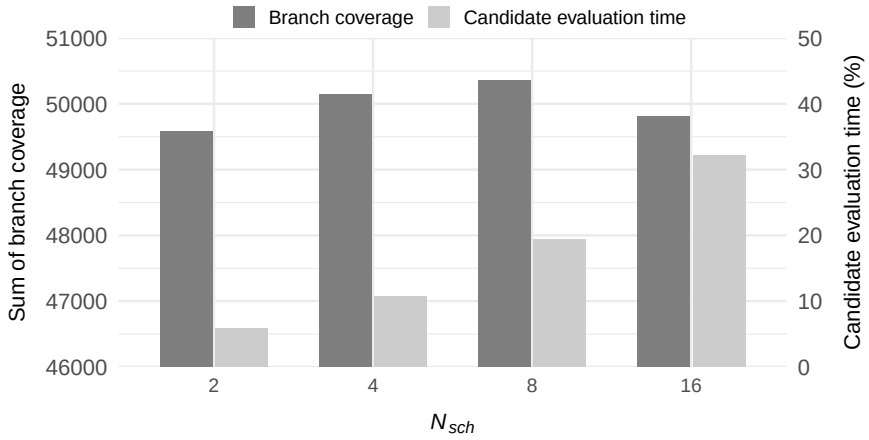


Figure 3: Impact of N_{sch} on the total branch coverage and the average proportion of the fuzzing time spent by SPAFL++ on candidate evaluation. The dark bars refer to the left axis and the light bars to the right axis.

6 Discussion

6.1 Threats to Validity

First, the generality of our benchmark suite can be a threat to validity, as we evaluated SPAFL++ on 9 programs. To mitigate this issue, we carefully selected programs that cover diverse input-processing behaviors and have been widely adopted in prior fuzzing research [5,9,39] and ensured that the size of our benchmark suite is in line with common practice in the field. According to Schloegel *et al.* [53], recent fuzzing papers used 8.9 programs on average in their evaluations. Nevertheless, our findings on the relation between bitmap coverage and splicing effectiveness may not generalize to program classes that are not represented by our benchmark, such as stateful network services or language interpreters.

Next, due to the inherent non-determinism of fuzzing, a single run of fuzzing may not faithfully reflect the effectiveness of our technique. To mitigate this, we repeated each fuzzing campaign 20 times for 24 hours and assessed the statistical significance of the observed improvements using a one-sided Mann–Whitney U test, as recommended by prior work on fuzzing evaluation [46,53].

Our empirical analysis in Section 3.2 is additionally subject to the cumulative nature of coverage-guided fuzzing, in which the order of evaluating splicing pairs affects the measured effectiveness. We controlled for this by evaluating all ordered pairs under the same coverage state.

We used branch coverage and the number of test cases deemed interesting by AFL++, as proxies for fuzzing effectiveness. While code coverage is the standard metric of coverage-guided fuzzing, it is only a proxy for the ultimate goal of finding bugs. To reduce this gap, we additionally report the number of unique bugs, counted using the patch-based triage method suggested by Klees *et al.* [46].

6.2 Limitations and Future Work

Our splicing target-selection strategy relies on the coverage bitmap size of a candidate seed, which we found to be the most consistent indicator of splicing effectiveness among the features examined in Section 3.2. This choice, however, does not preclude the existence of features that predict a promising splicing target more accurately, nor the possibility of combining multiple features to improve the selection strategy. Systematically exploring additional features, and possibly combining several of them into a stronger selection criterion, is a promising direction for future work.

In our current implementation, the number of candidates examined per splicing stage is governed by a fixed parameter, N_{sch}, whose optimal value reflects a trade-off between the cost of searching for a good target and the budget left for havoc mutation (Section 5.4). Determining this budget adaptively (e.g., per program or per seed), rather than using a single static value, is left as future work.

Our design is orthogonal to prior work that improves where to splice by confining splice points to structural boundaries [12,38], and combining our strategy for splicing target selection with structure-aware splice positions is a natural next step that may yield further gains.

7 Conclusion

In this paper, we proposed SPAFL++, a coverage-guided grey-box fuzzer that improves the splicing stage of AFL++ by selecting promising target seeds and adaptively scheduling energy for the spliced seeds. By choosing splicing targets based on their coverage bitmap size and evaluating candidate spliced seeds to retain the most promising one, SPAFL++ combines seeds more effectively than the conventional splicing stage, which selects the target at random and reuses the original seed's energy. Our experimental results show that, over 9 benchmark programs, SPAFL++ covered 1.64% more branches than the AFL++ baseline while detecting the same or a greater number of unique bugs, demonstrating that carefully redesigning the long-overlooked splicing stage can improve the effectiveness of grey-box fuzzing.

Acknowledgement: We thank Jin Jeon, Hyunjung Go, Jaeyong Lee, Yewon Yoon, and Junghwan Kim for their assistance in implementing the fuzzer evaluation framework used in this study.

Funding Statement: This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (No.2026-0-00036, AI university (Sogang University)).

Author Contributions: The authors confirm contribution to the paper as follows: Conceptualization, Soomin Kim and Jaeseung Choi; methodology, YoungIn Ahn and Jaeseung Choi; software, YoungIn Ahn, Minseok Lee and Jaeseung Choi; validation, YoungIn Ahn and Minseok Lee; writing—original draft preparation, YoungIn Ahn, Minseok Lee, Soomin Kim and Jaeseung Choi; writing—review and editing, Soomin Kim and Jaeseung Choi; visualization, YoungIn Ahn and Minseok Lee; supervision, Soomin Kim and Jaeseung Choi. All authors reviewed and approved the final version of the manuscript.

Availability of Data and Materials: The code and data that support the findings of this study are openly available in GitHub at https://github.com/infosec-sogang/SPAFLPP and https://github.com/infosec-sogang/Splicing-Artifact.

Ethics Approval: Not applicable.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Miller BP, Fredriksen L, So B. An Empirical Study of the Reliability of UNIX Utilities. Communications of the ACM. 1990;33(12):32-44.
2. Manès VJM, Han H, Han C, Cha SK, Egele M, Schwartz EJ, et al. The Art, Science, and Engineering of Fuzzing: A Survey. IEEE Transactions on Software Engineering. 2021;47(11):2312-31.
3. Zalewski M. American Fuzzy Lop;. http://lcamtuf.coredump.cx/afl/.
4. OSS-Fuzz: Continuous Fuzzing for Open Source Software;. https://github.com/google/oss-fuzz.
5. Böhme M, Pham VT, Roychoudhury A. Coverage-based Greybox Fuzzing as Markov Chain. In: Proceedings of the ACM Conference on Computer and Communications Security; 2016. p. 1032-43.

6. Wang Y, Jia X, Liu Y, Zeng K, Bao T, Wu D, et al. Not All Coverage Measurements Are Equal: Fuzzing by Coverage Accounting for Input Prioritization. In: Proceedings of the Network and Distributed System Security Symposium; 2020. doi:10.14722/ndss.2020.24422.
7. Yue T, Wang P, Tang Y, Wang E, Yu B, Lu K, et al. EcoFuzz: Adaptive Energy-Saving Greybox Fuzzing as a Variant of the Adversarial Multi-Armed Bandit. In: Proceedings of the USENIX Security Symposium; 2020. p. 2307-24.
8. You W, Liu X, Ma S, Perry D, Zhang X, Liang B. SLF: Fuzzing Without Valid Seed Inputs. In: Proceedings of the International Conference on Software Engineering; 2019. p. 712-23.
9. Zheng H, Toffalini F, Böhme M, Payer M. MendelFuzz: The Return of the Deterministic Stage. In: Proceedings of the International Symposium on Foundations of Software Engineering; 2025. p. 44-64.
10. Nagy S, Nguyen-Tuong A, Hiser JD, Davidson JW, Hicks M. Same Coverage, Less bloat: Accelerating Binary-only Fuzzing with Coverage-preserving Coverage-guided tracing. In: Proceedings of the ACM Conference on Computer and Communications Security; 2021. p. 351-65.
11. Xu W, Kashyap S, Min C, Kim T. Designing New Operating Primitives to Improve Fuzzing Performance. In: Proceedings of the ACM Conference on Computer and Communications Security; 2017. p. 2313-28.
12. Pham VT, Böhme M, Santosa AE, Căciulescu AR, Roychoudhury A. Smart greybox fuzzing. IEEE Transactions on Software Engineering. 2019;47(9):1980-97.
13. Aschermann C, Jauernig P, Frassetto T, Sadeghi AR, Holz T, Teuchert D. NAUTILUS: Fishing for Deep Bugs with Grammars. In: Proceedings of the Network and Distributed System Security Symposium; 2019. doi:10.14722/ndss.2019.23412.
14. Blazytko T, Aschermann C, Schlögel M, Abbasi A, Schumilo S, Wörner S, et al. GRIMOIRE: Synthesizing structure while fuzzing. In: Proceedings of the USENIX Security Symposium; 2019. p. 1985-2002.
15. Lin M, Zeng Y, Wu T, Wang Q, Fang L, Guo S. GSA-Fuzz: Optimize Seed Mutation with Gravitational Search Algorithm. Security and Communication Networks. 2022;2022(1). doi:10.1155/2022/1505842.
16. Rebert A, Cha SK, Avgerinos T, Foote J, Warren D, Grieco G, et al. Optimizing Seed Selection for Fuzzing. In: Proceedings of the USENIX Security Symposium; 2014. p. 861-75.
17. Godefroid P, Levin MY, Molnar DA. Automated Whitebox Fuzz Testing. In: Proceedings of the Network and Distributed System Security Symposium; 2008. p. 151-66.
18. Lyu C, Ji S, Zhang C, Li Y, Lee WH, Song Y, et al. MOpt: Optimized Mutation Scheduling for Fuzzers. In: Proceedings of the USENIX Security Symposium; 2019. p. 1949-66.
19. Kitsios K, Böhme M, Bacchelli A. On Interaction Effects in Greybox Fuzzing. In: Proceedings of the International Conference on Software Engineering; 2026. p. 3599-610.
20. Böhme M, Pham VT, Nguyen MD, Roychoudhury A. Directed Greybox Fuzzing. In: Proceedings of the ACM Conference on Computer and Communications Security; 2017. p. 2329-44.
21. Böhme M, Manès VJM, Cha SK. Boosting Fuzzer Efficiency: An Information Theoretic Perspective. In: Proceedings of the International Symposium on Foundations of Software Engineering; 2020. p. 678-89.
22. Li Y, Xue Y, Chen H, Wu X, Zhang C, Xie X, et al. Cerebro: Context-Aware Adaptive Fuzzing for Effective Vulnerability Detection. In: Proceedings of the International Symposium on Foundations of Software Engineering; 2019. p. 533-44.
23. Wang J, Song C, Yin H. Reinforcement Learning-based Hierarchical Seed Scheduling for Greybox Fuzzing. In: Proceedings of the Network and Distributed System Security Symposium; 2021. doi:10.14722/ndss.2021.24486.
24. She D, Shah A, Jana S. Effective seed scheduling for fuzzing with graph centrality analysis. In: Proceedings of the IEEE Symposium on Security and Privacy; 2022. p. 2194-211.
25. Qian R, Fang C, Chen Z, Fu Y, Chen Z. Peeling Off the Cocoon: Unveiling Suppressed Golden Seeds for Mutational Greybox Fuzzing. In: Proceedings of the ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications; 2026. p. 143-69.
26. Jauernig P, Jakobovic D, Picek S, Stapf E, Sadeghi AR. DARWIN: Survival of the Fittest Fuzzing Mutators. In: Proceedings of the Network and Distributed System Security Symposium; 2023. doi:10.14722/ndss.2023.23159.
27. Wu M, Jiang L, Xiang J, Huang Y, Cui H, Zhang L, et al. One Fuzzing Strategy to Rule Them All. In: Proceedings of the International Conference on Software Engineering; 2022. p. 1634-45.
28. Lee M, Cha S, Oh H. Learning Seed-Adaptive Mutation Strategies for Greybox Fuzzing. In: Proceedings of the International Conference on Software Engineering; 2023. p. 384-96.

29. Yu D, Wang Y, Zhang C, Lan Y, Jiang Z, Gan S, et al. xFUZZ: A Flexible Framework for Fine-Grained, Runtime-Adaptive Fuzzing Strategy Composition. In: Proceedings of the International Symposium on Software Testing and Analysis; 2025. p. 69-91.
30. Dong R, Wang Y, Zhang K, Yu D, Zhu X, Wang S, et al. TrioFuzz: A Three-Tier Architecture for Adaptive Strategy Selection in Fuzzing. In: Proceedings of the USENIX Security Symposium; 2026. p. 4031-47.
31. Wang Y, Yu D, Dong R, Chen Y, Zhu X, Zhang C. Bulbasaur: Branch-Guided Online Mutator Generation for Greybox Fuzzing. In: Proceedings of the USENIX Security Symposium; 2026. p. 4089-108.
32. Chen Y, Zhang C, Wang L, Zhu W, Luo C, Gui N, et al. IDFuzz: Intelligent Directed Grey-box Fuzzing. In: Proceedings of the USENIX Security Symposium; 2025. p. 6219-38.
33. Xie Y, Zhang W, She D. ZTaint-Havoc: From Havoc Mode to Zero-Execution Fuzzing-Driven Taint Inference. In: Proceedings of the International Symposium on Software Testing and Analysis; 2025. p. 917-39.
34. Chen P, Chen H. Angora: Efficient Fuzzing by Principled Search. In: Proceedings of the IEEE Symposium on Security and Privacy; 2018. p. 855-69.
35. Gan S, Zhang C, Qin X, Tu X, Li K, Pei Z, et al. CollAFL: Path Sensitive Fuzzing. In: Proceedings of the IEEE Symposium on Security and Privacy; 2018. p. 660-77.
36. Manès VJM, Kim S, Cha SK. Ankou: Guiding Grey-box Fuzzing towards Combinatorial Difference. In: Proceedings of the International Conference on Software Engineering; 2020. p. 1024-36.
37. Xiao J, Jiang P, Zhao Z, Huang R, Liu J, Li D. Robust, Efficient, and Widely Available Greybox Fuzzing for COTS Binaries with System Call Pattern Feedback. In: Proceedings of the USENIX Security Symposium; 2025. p. 6239-58.
38. Fioraldi A, D'Elia DC, Coppa E. WEIZZ: Automatic grey-box fuzzing for structured binary formats. In: Proceedings of the International Symposium on Software Testing and Analysis; 2020. p. 1-13.
39. Kukucka J, Pina L, Ammann P, Bell J. An empirical examination of fuzzer mutator performance. In: Proceedings of the International Symposium on Software Testing and Analysis; 2024. p. 1631-42.
40. Aschermann C, Schumilo S, Blazytko T, Gawlik R, Holz T. REDQUEEN: Fuzzing with Input-to-State Correspondence. In: Proceedings of the Network and Distributed System Security Symposium; 2019. doi: 10.14722/ndss.2019.23371.
41. Zhang X, Jiang J, Choi Y, Su T, Sun H, Wan C, et al. Understanding the Effectiveness of Mutators in Mutation-Based Protocol Fuzzing. In: Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering; 2026. p. 814-25.
42. Fioraldi A, Mantovani A, Maier D, Balzarotti D. Dissecting American Fuzzy Lop: A FuzzBench Evaluation. ACM Transactions on Software Engineering and Methodology. 2023;32(2):1-26.
43. Lemieux C, Padhye R, Sen K, Song D. PerfFuzz: Automatically Generating Pathological Inputs. In: Proceedings of the International Symposium on Software Testing and Analysis; 2018. p. 254-65.
44. Metzman J, Szekeres L, Simon L, Sprabery R, Arya A. FuzzBench: An Open Fuzzer Benchmarking Platform and Service. In: Proceedings of the International Symposium on Foundations of Software Engineering; 2021. p. 1393-403.
45. Lemieux C, Sen K. FairFuzz: A Targeted Mutation Strategy for Increasing Greybox Fuzz Testing Coverage. In: Proceedings of the International Conference on Automated Software Engineering; 2018. p. 475-85.
46. Klees G, Ruef A, Cooper B, Wei S, Hicks M. Evaluating fuzz testing. In: Proceedings of the ACM Conference on Computer and Communications Security; 2018. p. 2123-38.
47. Jiang L, Yuan H, Wu M, Zhang L, Zhang Y. Evaluating and Improving Hybrid Fuzzing. In: Proceedings of the International Conference on Software Engineering; 2023. p. 410-22.
48. Deng P, Yang Z, Zhang L, Yang G, Hong W, Zhang Y, et al. NestFuzz: Enhancing Fuzzing with Comprehensive Understanding of Input Processing Logic. In: Proceedings of the ACM Conference on Computer and Communications Security; 2023. p. 1272-86.
49. Fioraldi A, Maier D, Eißfeldt H, Heuse M. AFL++: Combining Incremental Steps of Fuzzing Research. In: Proceedings of the USENIX Workshop on Offensive Technologies; 2020. .
50. American Fuzzy Lop Plus Plus (AFL++);. https://github.com/aflplusplus/aflplusplus.
51. Xie Y, Zhang K, Liu Y, Yang R, Chen P, Wang S, et al. RandSet: Randomized Corpus Reduction for Fuzzing Seed Scheduling. In: Proceedings of the ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications; 2026. p. 1570-98.

52. Vargha A, Delaney HD. A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. Journal of Educational and Behavioral Statistics. 2000;25(2):101-32.
53. Schloegel M, Bars N, Schiller N, Bernhard L, Scharnowski T, Crump A, et al. SoK: Prudent Evaluation Practices for Fuzzing. In: Proceedings of the IEEE Symposium on Security and Privacy; 2024. p. 1974-93.